CoolSense: Simulated Sensors → Digital Twin → Neural Network Decision System

MVP System Design Document

---

1. SYSTEM OVERVIEW

CoolSense is a software system that:

1. **Ingests** real-time water quality data from sensors at multiple points in a cooling tower loop (for the MVP, these sensors are simulated — the software is identical whether the data comes from real hardware or a simulator)
2. **Models** the cooling tower as a digital twin, using multi-point data to identify where and why contamination occurs
3. **Decides** what operational parameters to change to minimize pollution in the blowdown discharge, using a neural network optimizer

The architecture is hardware-agnostic: the sensor interface is a simple data stream (JSON over WebSocket or a Python function call). When real sensors are connected later, you swap the simulator module for a sensor-reading module. Everything else stays the same.

---

2. THE COOLING TOWER LOOP — WHERE POLLUTION COMES FROM

To understand the system, you need to understand what happens to water in a cooling tower:

```
┌──────────────────────────┐
│ COOLING TOWER            │
│ (evaporation +           │
│  air scrubbing)          │
│                          │
│ Introduces:              │
│ - Airborne dust          │
│ - Microplastics          │
│ - Biological growth      │
│ - Concentration of       │
│   ALL dissolved          │
│   substances via         │
│   evaporation            │
└──────────┬───────────────┘
           │
           ▼
┌──────────────────────────┐
│ BASIN / SUMP             │
│                          │
│ Where chemicals are      │◄──── CHEMICAL INJECTION POINT
│ added:                   │      (biocides, inhibitors,
│ - Biocides (chlorine,    │       pH adjusters)
│   quats, bromine)        │
│ - Corrosion inhibitors   │      Introduces:
│   (phosphonates, zinc)   │       - Biocide residuals
│ - Scale inhibitors       │       - Phosphorus (from phosphonates)
│ - pH adjusters           │       - Zinc (from zinc-based inhibitors)
└──────────┬───────────────┘
           │
           ▼
┌──────────────────────────┐
│ RECIRCULATION PIPING     │
│ + HEAT EXCHANGERS        │
│                          │
│ Introduces:              │
│ - Dissolved copper       │
│   (from Cu tubes)        │
│ - Dissolved nickel       │
│   (from Ni alloys)       │
│ - Dissolved zinc         │
│   (from galvanized       │
│    sections)             │
│ - Scale particles        │
│ - Corrosion products     │
│   (turbidity)            │
└──────────┬───────────────┘
```

```
        │
        ▼
┌──────────────────────────────┐     ┌──────────────────────────────┐
│  BLOWDOWN LINE               │────▶│  DISCHARGE TO                │
│  (water dumped to            │     │  RECEIVING WATER             │
│   control concentration)     │     │  (river, lake,               │
│                              │     │   municipal sewer)           │
└──────────────────────────────┘     └──────────────────────────────┘
        ▲
        │
┌──────────────────────────────┐
│  MAKEUP WATER INTAKE         │
│  (fresh water added to       │
│   replace evaporation        │
│   + blowdown losses)         │
│                              │
│  Introduces:                 │
│  - Background minerals       │
│  - Background metals         │
│  - Municipal treatment       │
│    residuals                 │
└──────────────────────────────┘
```

The key diagnostic insight of multi-point sensing: If conductivity jumps between the makeup intake and the basin, that's concentration from evaporation. If pH drops between the basin and post-heat-exchanger, that's CO_2 absorption or acid corrosion. If turbidity spikes post-heat-exchanger but not pre, that's corrosion products from the piping. By comparing readings at different points, you isolate the source.

---

3. SENSOR SIMULATION — WHAT WE SIMULATE AND WHY

3.1 The 4 Sensor Nodes

We simulate sensor nodes at 4 strategic points in the loop. Each node reports 3 parameters.

Node 1: MAKEUP WATER INTAKE

Purpose: Baseline. Everything here is "what came in" — not the tower's fault. **What it tells the digital twin:** Input water quality.

Node 2: BASIN (POST-CHEMICAL INJECTION)

Purpose: Measures evaporative concentration + chemical addition effects. **What it tells the digital twin:**

- Conductivity here vs. Node 1 = cycles of concentration (CoC)
- pH shift from chemical treatment
- Turbidity from airborne debris + precipitated solids

Node 3: POST-HEAT EXCHANGER (RETURN LINE)

Purpose: Measures what the piping and heat exchangers added. **What it tells the digital twin:**

- Conductivity increase = dissolved metals from corrosion
- pH drop = CO_2 or acid corrosion
- Turbidity spike = corrosion particles, scale fragments

Node 4: BLOWDOWN LINE

Purpose: Exactly what's being discharged. The "final answer." **What it tells the digital twin:** Cumulative effect of all upstream processes.

3.2 Parameters Per Node

Each node reports the same 3 values:

Sensor	What it measures	Why it matters	Typical range
pH	Hydrogen ion concentration	Controls metal solubility, biocide effectiveness, scaling tendency	6.5 – 9.5
Conductivity (μS/cm)	Proxy for total dissolved solids	Tells you how concentrated the water is; ratio of Node 2/Node 1 = cycles of concentration	200 – 10,000
Turbidity (NTU)	Suspended solids / cloudiness	Catch-all for corrosion products, particles, biological growth	0 – 500

3.3 The Simulator Module

The simulator generates realistic, time-varying sensor data that behaves like a real cooling tower. It models drift, events, noise, and the physical coupling between nodes.

```python
```

```python
# simulator.py — Generates realistic multi-node cooling tower sensor data

import time
import math
import random
import threading

class CoolingTowerSimulator:
    """
    Simulates 4 sensor nodes in a cooling tower loop.
    Produces realistic, physically-coupled, time-varying data.

    The simulator models:
    - Baseline conditions (steady-state operation)
    - Gradual drift (evaporation concentrating water over hours)
    - Random events (corrosion spikes, biocide dosing, weather changes)
    - Noise (sensor measurement uncertainty)
    - Physical coupling (downstream nodes respond to upstream changes)
    """

    def __init__(self):
        # --- Operational parameters (these are what the NN will optimize) ---
        self.blowdown_rate = 50.0        # m³/day (control variable)
        self.biocide_dose = 2.0          # mg/L as Cl₂ (control variable)
        self.target_ph = 8.0             # pH setpoint (control variable)
        self.target_coc = 5.0            # cycles of concentration target (control variable)

        # --- Environmental / fixed conditions ---
        self.ambient_temp = 35.0         # °C
        self.makeup_ph = 7.8
        self.makeup_ec = 500.0           # µS/cm
        self.makeup_turbidity = 2.0      # NTU

        # --- Internal state (evolves over time) ---
        self.current_coc = 5.0
        self.corrosion_severity = 0.1    # 0 = none, 1 = severe
        self.biofouling_level = 0.05     # 0 = none, 1 = severe
        self.time_since_biocide = 0.0    # hours
        self.biocide_residual = 1.5      # mg/L

        # --- Event system ---
        self._events = []
        self._lock = threading.Lock()
        self._start_time = time.time()

    def elapsed_hours(self):
```

```
        return (time.time() - self._start_time) / 3600.0

    def inject_event(self, event_type, magnitude=1.0):
        """
        Inject a simulated event for demo purposes.

        event_type options:
        - 'corrosion_spike': sudden increase in heat exchanger corrosion
        - 'biocide_overdose': biocide concentration jumps
        - 'biocide_underdose': biocide drops, biofouling risk
        - 'makeup_quality_drop': incoming water quality degrades (drought)
        - 'concentration_buildup': CoC drifts upward (blowdown valve stuck)
        - 'ph_drop': acid event (CO₂ ingress or chemical spill)
        - 'turbidity_event': particulate surge (construction upstream, storm)
        """
        with self._lock:
            self._events.append({
                'type': event_type,
                'magnitude': magnitude,
                'start_time': time.time(),
                'duration': random.uniform(30, 120),  # seconds
            })

    def _process_events(self):
        """Update internal state based on active events."""
        now = time.time()
        with self._lock:
            active_events = [e for e in self._events
                             if now - e['start_time'] < e['duration']]
            self._events = active_events

        for event in active_events:
            progress = (now - event['start_time']) / event['duration']
            # Events ramp up then fade
            intensity = math.sin(progress * math.pi) * event['magnitude']

            if event['type'] == 'corrosion_spike':
                self.corrosion_severity = min(1.0, 0.1 + intensity * 0.8)
            elif event['type'] == 'biocide_overdose':
                self.biocide_residual = min(10.0, 2.0 + intensity * 5.0)
            elif event['type'] == 'biocide_underdose':
                self.biocide_residual = max(0.0, 1.5 - intensity * 1.5)
                self.biofouling_level = min(1.0, 0.05 + intensity * 0.4)
            elif event['type'] == 'makeup_quality_drop':
                self.makeup_ec = 500 + intensity * 300
                self.makeup_turbidity = 2.0 + intensity * 15.0
            elif event['type'] == 'concentration_buildup':
```

```
            self.current_coc = min(10, self.target_coc + intensity * 3)
        elif event['type'] == 'ph_drop':
            self.target_ph = max(6.5, 8.0 - intensity * 1.5)
        elif event['type'] == 'turbidity_event':
            self.makeup_turbidity = 2.0 + intensity * 50.0

    def _add_noise(self, value, noise_pct=0.02):
        """Add realistic sensor noise."""
        return value * (1 + random.gauss(0, noise_pct))

    def read_all_nodes(self):
        """
        Returns current sensor readings for all 4 nodes.
        This is the ONLY interface the rest of the system uses.
        When real hardware is connected, replace this function.
        """
        self._process_events()

        # Slow drift: CoC gradually approaches target
        self.current_coc += (self.target_coc - self.current_coc) * 0.01

        # Biocide decays over time
        self.time_since_biocide += 2.0 / 3600.0  # 2-second tick
        decay_rate = 0.2 * (1.05 ** (self.ambient_temp - 25))
        self.biocide_residual = self.biocide_dose * math.exp(
            -decay_rate * self.time_since_biocide
        )
        # Periodic re-dosing every 4 hours
        if self.time_since_biocide > 4.0:
            self.time_since_biocide = 0.0
            self.biocide_residual = self.biocide_dose

        # Corrosion slowly returns to baseline if no event
        self.corrosion_severity = max(0.05,
            self.corrosion_severity * 0.995 + 0.05 * 0.005)

        # --- NODE 1: Makeup Water ---
        node1 = {
            'ph': self._add_noise(self.makeup_ph),
            'ec': self._add_noise(self.makeup_ec),
            'turbidity': self._add_noise(self.makeup_turbidity, 0.05),
        }

        # --- NODE 2: Basin (post-chemical injection) ---
        basin_ec = self.makeup_ec * self.current_coc
        basin_ph = self.target_ph + random.gauss(0, 0.05)
        basin_turbidity = (self.makeup_turbidity * self.current_coc * 0.3
```

```
                + self.biofouling_level * 20
                + random.gauss(0, 1))
basin_turbidity = max(0, basin_turbidity)

node2 = {
    'ph': self._add_noise(basin_ph),
    'ec': self._add_noise(basin_ec),
    'turbidity': self._add_noise(basin_turbidity, 0.05),
}

# --- NODE 3: Post-Heat Exchanger ---
# Corrosion adds conductivity (dissolved metals) and turbidity (particles)
corrosion_ec_add = self.corrosion_severity * 80
corrosion_turb_add = self.corrosion_severity * 60
# pH drops slightly from corrosion reactions
corrosion_ph_drop = self.corrosion_severity * 0.3

posthx_ec = basin_ec + corrosion_ec_add
posthx_ph = basin_ph - corrosion_ph_drop
posthx_turbidity = basin_turbidity + corrosion_turb_add

node3 = {
    'ph': self._add_noise(posthx_ph),
    'ec': self._add_noise(posthx_ec),
    'turbidity': self._add_noise(max(0, posthx_turbidity), 0.05),
}

# --- NODE 4: Blowdown ---
# Approximately equals Node 3 with minor additional changes
node4 = {
    'ph': self._add_noise(posthx_ph - 0.02),
    'ec': self._add_noise(posthx_ec + 5),
    'turbidity': self._add_noise(max(0, posthx_turbidity - 1), 0.05),
}

return {
    'node1': node1,
    'node2': node2,
    'node3': node3,
    'node4': node4,
    'timestamp': time.time(),
    # Expose internal state for validation (not used by twin/NN)
    '_internal': {
        'true_coc': self.current_coc,
        'true_corrosion': self.corrosion_severity,
        'true_biocide_residual': self.biocide_residual,
        'true_biofouling': self.biofouling_level,
```

```
        }
    }

    def apply_recommendations(self, recommendations):
        """
        Apply neural network recommendations to the simulated system.
        This closes the loop: NN recommends → system responds → sensors change.
        """
        if 'recommended_blowdown_rate_m3_day' in recommendations:
            self.blowdown_rate = recommendations['recommended_blowdown_rate_m3_day']
        if 'recommended_biocide_dose_mgL' in recommendations:
            self.biocide_dose = recommendations['recommended_biocide_dose_mgL']
        if 'recommended_ph_setpoint' in recommendations:
            self.target_ph = recommendations['recommended_ph_setpoint']
        if 'recommended_coc_target' in recommendations:
            self.target_coc = recommendations['recommended_coc_target']
```

How to use the simulator in the demo:

```python
sim = CoolingTowerSimulator()

# Normal operation — data just flows
data = sim.read_all_nodes()

# Trigger a dramatic event during the demo
sim.inject_event('corrosion_spike', magnitude=1.0)  # turbidity surges at Node 3

# After NN recommends changes, apply them and watch the system respond
sim.apply_recommendations(recommendations)
```

Swapping to real hardware later:

The only thing that changes is replacing `sim.read_all_nodes()` with a function that reads real I2C sensors. The digital twin, neural network, and dashboard code stay identical.

---

4. THE DIGITAL TWIN — WHAT IT IS AND HOW TO BUILD IT

4.1 What the Digital Twin Actually Is

The digital twin is a **mathematical model of the cooling tower water loop** that runs in real time alongside the data stream. It takes sensor readings as inputs and computes things that the 3 cheap sensors CANNOT directly measure — specifically:

- **Estimated dissolved metal concentrations** (Cu, Zn) based on pH and turbidity differentials

- **Estimated biocide residual** based on dosing schedule, pH, and decay kinetics
- **Estimated phosphorus load** from corrosion inhibitor dosing
- **Predicted blowdown composition** for the next 1–24 hours based on current trends
- **Identification of which loop segment** is causing each detected change
- **Compliance status** against permit limits

The digital twin turns 12 raw numbers (3 sensors × 4 nodes) into a complete picture of the cooling tower's chemistry.

4.2 The Math — 4 Sub-Models in Python

Sub-Model 1: Concentration Model (Mass Balance)

This is the foundation. Everything in cooling water concentrates as water evaporates.

```python
def concentration_model(node1_ec, node2_ec, makeup_chemistry):
    """
    Determines cycles of concentration and estimates basin chemistry.

    node1_ec: conductivity at makeup intake (μS/cm)
    node2_ec: conductivity at basin (μS/cm)
    makeup_chemistry: dict of {substance: concentration_mg_L} in makeup water

    Returns: estimated basin concentrations + CoC
    """
    coc = node2_ec / node1_ec if node1_ec > 0 else 1.0

    basin_chemistry = {}
    for substance, conc in makeup_chemistry.items():
        basin_chemistry[substance] = conc * coc

    return basin_chemistry, coc
```

What this means: If makeup water has 500 μS/cm and basin has 2500 μS/cm, you're at 5 cycles of concentration. Every dissolved substance in the makeup is now 5× more concentrated. That's how 10 ppb copper in tap water becomes 50 ppb in the basin — before corrosion even adds more.

Sub-Model 2: Corrosion / Metal Leaching Model

Estimates dissolved metal concentrations based on pH. The key relationship: metal solubility increases as pH decreases. The turbidity differential between nodes isolates where corrosion is occurring.

```python
```

```python
import math

def corrosion_model(node2_ph, node3_ph, node2_turb, node3_turb, temperature_C=35):
    """
    Estimates dissolved Cu/Zn concentrations and corrosion severity
    based on pH and turbidity differential between basin and post-heat-exchanger.

    The core science:
    - Copper solubility follows the Pourbaix diagram: lower pH = more dissolved Cu
    - Turbidity increase between Node 2 and Node 3 = corrosion particles from piping
    """
    # Cu solubility estimate from pH (simplified Pourbaix diagram)
    # log10(Cu_ppb) ≈ -0.5 × pH + 6.0 (valid for pH 7–9.5 in aerated water)
    ph = node3_ph  # use post-heat-exchanger pH (where corrosion happens)
    cu_ppb = 10 ** (-0.5 * ph + 6.0)
    cu_ppb = max(5, min(cu_ppb, 1000))  # clamp to realistic range

    # Temperature correction: corrosion rate roughly doubles per 20°C
    temp_factor = 2 ** ((temperature_C - 25) / 20)
    cu_ppb *= temp_factor

    # Zn estimate (similar relationship, different coefficients)
    zn_ppb = 10 ** (-0.4 * ph + 5.5)
    zn_ppb = max(10, min(zn_ppb, 500)) * temp_factor

    # Corrosion severity index from turbidity delta
    # If turbidity jumps between basin and post-HX, that's corrosion products
    if node2_turb > 0:
        corrosion_severity = (node3_turb - node2_turb) / node2_turb
    else:
        corrosion_severity = 0

    # Source diagnosis
    if corrosion_severity > 0.5:
        source_diagnosis = "SEVERE corrosion in heat exchanger / piping segment"
    elif corrosion_severity > 0.2:
        source_diagnosis = "Moderate corrosion in heat exchanger segment"
    elif corrosion_severity > 0.05:
        source_diagnosis = "Minor corrosion — within normal range"
    else:
        source_diagnosis = "No significant corrosion detected"

    return {
        'cu_ppb': round(cu_ppb, 1),
        'zn_ppb': round(zn_ppb, 1),
        'corrosion_severity_index': round(max(0, corrosion_severity), 3),
```

```
    'corrosion_flag': corrosion_severity > 0.2,
    'source_diagnosis': source_diagnosis,
}
```

Sub-Model 3: Biocide Decay Model

Biocides (chlorine, bromine) decay over time following first-order kinetics. The rate depends on temperature and pH. Critically, the *effective* biocide fraction depends on pH — chlorine shifts between effective HOCl and weak OCl^-.

```python
```

```python
def biocide_model(ph, temperature_C, dosing_rate_mgL, time_since_dose_hours):
    """
    Estimates current biocide residual and effective fraction.

    The core science:
    - Biocide decays exponentially: C(t) = C₀ × e^(-k×t)
    - Decay rate k increases with temperature and pH
    - Effective chlorine (HOCl) fraction drops sharply above pH 7.5
      At pH 7.0: ~75% effective | pH 8.0: ~25% | pH 8.5: ~10%
    """
    # Base decay rate at 25°C, pH 7
    k_base = 0.2  # per hour

    # Temperature correction
    k = k_base * (1.05 ** (temperature_C - 25))

    # pH correction (faster decay at higher pH)
    k *= 10 ** (0.1 * (ph - 7))

    # Current total chlorine residual
    total_cl2 = dosing_rate_mgL * math.exp(-k * time_since_dose_hours)

    # HOCl fraction — the part that actually kills bacteria
    # pKa of HOCl = 7.54
    hocl_fraction = 1 / (1 + 10 ** (ph - 7.54))
    effective_cl2 = total_cl2 * hocl_fraction

    # Diagnosis
    if effective_cl2 < 0.2:
        biocide_status = "INSUFFICIENT — biofouling risk"
    elif effective_cl2 < 0.5:
        biocide_status = "Marginal — monitor closely"
    elif total_cl2 > 5.0:
        biocide_status = "OVERDOSED — excess biocide will discharge in blowdown"
    else:
        biocide_status = "Adequate"

    return {
        'total_chlorine_mgL': round(total_cl2, 3),
        'effective_hocl_mgL': round(effective_cl2, 3),
        'hocl_fraction_pct': round(hocl_fraction * 100, 1),
        'decay_rate_per_hour': round(k, 3),
        'biocide_status': biocide_status,
    }
```

Sub-Model 4: Blowdown Discharge Predictor & Compliance Checker

Combines all sub-models to predict what leaves the facility in the blowdown and checks it against permit limits.

```python
```

```python
def blowdown_predictor(node4_readings, concentration_output, corrosion_output,
                       biocide_output, coc, blowdown_flow_m3_day, permit_limits):
    """
    Predicts blowdown discharge composition and flags permit exceedances.

    permit_limits example:
    {
        'cu_ppb': 18,             # EPA acute freshwater Cu criterion
        'zn_ppb': 120,            # EPA acute freshwater Zn criterion
        'total_chlorine_mgL': 0.1, # typical NPDES TRC limit
        'ph_min': 6.5,            # NPDES pH range
        'ph_max': 9.0,
        'turbidity_ntu': 50,      # typical turbidity limit
    }
    """
    discharge = {
        'ph': node4_readings['ph'],
        'conductivity_uScm': node4_readings['ec'],
        'turbidity_ntu': node4_readings['turbidity'],
        'cu_ppb': corrosion_output['cu_ppb'],
        'zn_ppb': corrosion_output['zn_ppb'],
        'total_chlorine_mgL': biocide_output['total_chlorine_mgL'],
        'cycles_of_concentration': coc,
    }

    # Mass loadings (how much contaminant per day)
    mass_loadings = {
        'cu_g_day': round(discharge['cu_ppb'] * 1e-6 * blowdown_flow_m3_day * 1e6, 3),
        'zn_g_day': round(discharge['zn_ppb'] * 1e-6 * blowdown_flow_m3_day * 1e6, 3),
        'chlorine_g_day': round(discharge['total_chlorine_mgL'] * blowdown_flow_m3_day, 1),
    }

    # Permit compliance check
    exceedances = []
    if discharge['cu_ppb'] > permit_limits.get('cu_ppb', 9999):
        exceedances.append({
            'parameter': 'Copper',
            'value': f"{discharge['cu_ppb']} ppb",
            'limit': f"{permit_limits['cu_ppb']} ppb",
            'severity': round((discharge['cu_ppb'] / permit_limits['cu_ppb'] - 1) * 100, 1),
        })
    if discharge['zn_ppb'] > permit_limits.get('zn_ppb', 9999):
        exceedances.append({
            'parameter': 'Zinc',
            'value': f"{discharge['zn_ppb']} ppb",
            'limit': f"{permit_limits['zn_ppb']} ppb",
```

```
        'severity': round((discharge['zn_ppb'] / permit_limits['zn_ppb'] - 1) * 100, 1),
    })
if discharge['total_chlorine_mgL'] > permit_limits.get('total_chlorine_mgL', 9999):
    exceedances.append({
        'parameter': 'Total Residual Chlorine',
        'value': f"{discharge['total_chlorine_mgL']} mg/L",
        'limit': f"{permit_limits['total_chlorine_mgL']} mg/L",
        'severity': round((discharge['total_chlorine_mgL'] / permit_limits['total_chlorine_mgL'] - 1) * 100, 1),
    })
if discharge['turbidity_ntu'] > permit_limits.get('turbidity_ntu', 9999):
    exceedances.append({
        'parameter': 'Turbidity',
        'value': f"{discharge['turbidity_ntu']} NTU",
        'limit': f"{permit_limits['turbidity_ntu']} NTU",
        'severity': round((discharge['turbidity_ntu'] / permit_limits['turbidity_ntu'] - 1) * 100, 1),
    })
if discharge['ph'] < permit_limits.get('ph_min', 0):
    exceedances.append({
        'parameter': 'pH (low)',
        'value': f"{discharge['ph']:.2f}",
        'limit': f"{permit_limits['ph_min']}",
        'severity': round((permit_limits['ph_min'] - discharge['ph']) * 100, 1),
    })
if discharge['ph'] > permit_limits.get('ph_max', 14):
    exceedances.append({
        'parameter': 'pH (high)',
        'value': f"{discharge['ph']:.2f}",
        'limit': f"{permit_limits['ph_max']}",
        'severity': round((discharge['ph'] - permit_limits['ph_max']) * 100, 1),
    })

return {
    'discharge_composition': discharge,
    'mass_loadings_per_day': mass_loadings,
    'exceedances': exceedances,
    'n_exceedances': len(exceedances),
    'compliant': len(exceedances) == 0,
}
```

4.3 Technology Stack

Component	Technology	Why
Simulator	Pure Python (see above)	No dependencies, runs anywhere
Sub-models	Pure Python (`math` only)	Fast, no heavy libraries

Component	Technology	Why
Data storage	SQLite or in-memory list	Stores history for NN and charts
Dashboard backend	Flask or FastAPI	Serves API + WebSocket
Dashboard frontend	Streamlit (fastest) or React + Recharts (prettiest)	See Section 6
Neural network	PyTorch	Lightweight, well-documented

5. THE NEURAL NETWORK — WHAT IT DECIDES AND HOW

5.1 What the Neural Network Optimizes

The NN solves a **multi-objective optimization problem:**

Minimize: Total contaminant mass in blowdown (Cu + Zn + biocide residual + phosphorus)

Subject to:

- Cooling performance must remain adequate (CoC within acceptable range)
- Biocide residual must stay above minimum effective level (microbial control)
- pH must stay within range that limits both corrosion AND scaling
- Blowdown volume shouldn't be excessive (water waste)

By adjusting these 4 control variables:

1. **Blowdown rate** (higher = dilutes contaminants but wastes water)
2. **Biocide dose** (lower = less biocide in discharge but risk of biofouling)
3. **Target pH setpoint** (higher = less corrosion/metal leaching but less effective biocide and more scaling risk)
4. **Cycles of concentration target** (lower = cleaner blowdown but more water consumption)

These four knobs interact in complex ways — that's why a neural network is better than simple rules.

5.2 Network Architecture

A small feedforward network. The physics are handled by the digital twin — the NN just learns the tradeoff surface between competing objectives.

```
INPUT LAYER (16 neurons):
├── Node 1: pH, EC, turbidity ──────────────────────── (3) raw sensor data
├── Node 2: pH, EC, turbidity ──────────────────────── (3) raw sensor data
├── Node 3: pH, EC, turbidity ──────────────────────── (3) raw sensor data
├── Node 4: pH, EC, turbidity ──────────────────────── (3) raw sensor data
├── Derived from digital twin:
│   ├── CoC ──────────────────────────────────────────────────── (1)
│   ├── Corrosion severity index ──────────────────── (1)
│   ├── Estimated Cu (ppb) ──────────────────────────── (1)
│   └── Estimated biocide residual (mg/L) ──────── (1)

HIDDEN LAYER 1: 32 neurons, ReLU activation
HIDDEN LAYER 2: 16 neurons, ReLU activation

OUTPUT LAYER (4 neurons, sigmoid → scaled to physical ranges):
├── Recommended blowdown rate ─────── 10–100 m³/day
├── Recommended biocide dose ──────── 0.5–5.0 mg/L
├── Recommended pH setpoint ────────── 7.5–9.0
└── Recommended CoC target ─────────── 3–8 cycles
```

5.3 Training: The Digital Twin Generates All Training Data

This is the key insight: **you don't need real-world data.** The digital twin acts as a simulator. You generate thousands of random scenarios, run the twin for each one, find the optimal controls by grid search, and train the NN on those pairs.

```python
```

```python
# neural_network.py — Training and inference for the CoolSense optimizer

import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import math


class CoolSenseOptimizer(nn.Module):
    def __init__(self):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(16, 32),
            nn.ReLU(),
            nn.Linear(32, 16),
            nn.ReLU(),
            nn.Linear(16, 4),
            nn.Sigmoid()  # outputs in [0, 1], scaled to parameter ranges
        )
        # Output scaling
        self.output_ranges = {
            'blowdown': (10.0, 100.0),
            'biocide': (0.5, 5.0),
            'ph': (7.5, 9.0),
            'coc': (3.0, 8.0),
        }

    def forward(self, x):
        raw = self.network(x)
        scaled = torch.zeros_like(raw)
        for i, key in enumerate(self.output_ranges):
            lo, hi = self.output_ranges[key]
            scaled[:, i] = raw[:, i] * (hi - lo) + lo
        return scaled

    def forward_raw(self, x):
        """Returns raw sigmoid outputs (for training against normalized targets)."""
        return self.network(x)


def compute_pollution_score(target_ph, biocide_dose, target_coc, makeup_ec,
                            ambient_temp, blowdown_rate):
    """
    Given a set of control parameters and conditions, compute a pollution score.
    Lower = less pollution = better.
```

```
Uses the same physics as the digital twin sub-models.
"""
# Copper solubility from pH
cu_ppb = 10 ** (-0.5 * target_ph + 6.0)
temp_factor = 2 ** ((ambient_temp - 25) / 20)
cu_ppb *= temp_factor

# Zinc
zn_ppb = 10 ** (-0.4 * target_ph + 5.5) * temp_factor

# Biocide in blowdown (after 2 hours average residence)
k = 0.2 * (1.05 ** (ambient_temp - 25)) * (10 ** (0.1 * (target_ph - 7)))
residual_biocide = biocide_dose * math.exp(-k * 2.0)

# Effective biocide check
hocl_fraction = 1 / (1 + 10 ** (target_ph - 7.54))
effective_biocide = residual_biocide * hocl_fraction

# Basin conductivity
basin_ec = makeup_ec * target_coc

# --- Pollution score components (each normalized to ~0-1 scale) ---
cu_score = cu_ppb / 18.0          # 18 ppb = permit limit
zn_score = zn_ppb / 120.0         # 120 ppb = permit limit
biocide_score = residual_biocide / 0.1  # 0.1 mg/L = permit limit
water_waste = blowdown_rate / 100.0     # higher = more waste

# --- Constraint penalties ---
penalty = 0.0
if effective_biocide < 0.2:
    penalty += 10.0  # must maintain microbial control
if basin_ec > 6000:
    penalty += 5.0   # excessive concentration risks scaling
if target_ph < 7.0 or target_ph > 9.5:
    penalty += 5.0   # pH out of operational range
if cu_ppb > 200:
    penalty += 3.0   # severely exceeding Cu limits

total = cu_score + zn_score + biocide_score + water_waste + penalty
return total, {
    'cu_ppb': cu_ppb,
    'zn_ppb': zn_ppb,
    'residual_biocide': residual_biocide,
    'effective_biocide': effective_biocide,
}
```

```python
def generate_training_data(n_scenarios=5000):
    """
    Generate (sensor_state → optimal_controls) training pairs
    by running the digital twin physics across random scenarios.
    """
    X = []  # input states
    Y = []  # optimal control outputs

    for _ in range(n_scenarios):
        # --- Random scenario (environmental conditions) ---
        makeup_ec = np.random.uniform(300, 800)
        makeup_ph = np.random.uniform(7.0, 8.5)
        makeup_turb = np.random.uniform(1, 10)
        ambient_temp = np.random.uniform(20, 45)

        # Random current system state
        current_coc = np.random.uniform(3, 8)
        corrosion_severity = np.random.uniform(0, 0.8)

        # Compute what sensors would read at each node
        basin_ec = makeup_ec * current_coc
        basin_ph = makeup_ph + np.random.uniform(-0.3, 0.5)
        basin_turb = makeup_turb * current_coc * 0.3 + np.random.uniform(0, 10)

        posthx_ec = basin_ec + corrosion_severity * 80
        posthx_ph = basin_ph - corrosion_severity * 0.3
        posthx_turb = basin_turb + corrosion_severity * 60

        blowdown_ec = posthx_ec + np.random.uniform(0, 10)
        blowdown_ph = posthx_ph - 0.02
        blowdown_turb = max(0, posthx_turb - np.random.uniform(0, 2))

        # Derived values (what digital twin computes)
        est_cu = 10 ** (-0.5 * posthx_ph + 6.0) * (2 ** ((ambient_temp - 25) / 20))
        est_biocide = 2.0 * math.exp(-0.2 * 2)  # approximate

        # Build 16-feature input vector
        state_vector = [
            makeup_ph, makeup_ec, makeup_turb,
            basin_ph, basin_ec, basin_turb,
            posthx_ph, posthx_ec, posthx_turb,
            blowdown_ph, blowdown_ec, blowdown_turb,
            current_coc,
            corrosion_severity,
            min(est_cu, 1000),
            est_biocide,
        ]
```

```
        # --- Grid search for optimal controls ---
        best_score = float('inf')
        best_controls = [50, 2.0, 8.0, 5.0]  # defaults

        for blowdown_rate in np.linspace(10, 100, 10):
            for biocide_dose in np.linspace(0.5, 5.0, 10):
                for target_ph in np.linspace(7.5, 9.0, 8):
                    for target_coc in np.linspace(3, 8, 6):
                        score, _ = compute_pollution_score(
                            target_ph, biocide_dose, target_coc,
                            makeup_ec, ambient_temp, blowdown_rate
                        )
                        if score < best_score:
                            best_score = score
                            best_controls = [blowdown_rate, biocide_dose,
                                             target_ph, target_coc]

        X.append(state_vector)
        Y.append(best_controls)

    return np.array(X, dtype=np.float32), np.array(Y, dtype=np.float32)


def train_model(n_scenarios=5000, n_epochs=200):
    """Train the neural network using digital-twin-generated data."""

    print(f"Generating {n_scenarios} training scenarios from digital twin...")
    X, Y = generate_training_data(n_scenarios)

    # Normalize inputs (zero mean, unit variance)
    X_mean = X.mean(axis=0)
    X_std = X.std(axis=0) + 1e-8
    X_norm = (X - X_mean) / X_std

    # Normalize outputs to [0, 1] for sigmoid targets
    output_ranges = [(10, 100), (0.5, 5.0), (7.5, 9.0), (3, 8)]
    Y_norm = np.zeros_like(Y)
    for i, (lo, hi) in enumerate(output_ranges):
        Y_norm[:, i] = (Y[:, i] - lo) / (hi - lo)

    X_tensor = torch.FloatTensor(X_norm)
    Y_tensor = torch.FloatTensor(Y_norm)

    model = CoolSenseOptimizer()
    optimizer = optim.Adam(model.parameters(), lr=0.001)
    loss_fn = nn.MSELoss()
```

```
    print(f"Training for {n_epochs} epochs...")
    for epoch in range(n_epochs):
        pred = model.forward_raw(X_tensor)
        loss = loss_fn(pred, Y_tensor)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if epoch % 25 == 0:
            print(f"  Epoch {epoch:3d}: loss = {loss.item():.6f}")

    # Save everything needed for inference
    checkpoint = {
        'model_state': model.state_dict(),
        'X_mean': X_mean,
        'X_std': X_std,
    }
    torch.save(checkpoint, 'coolsense_model.pt')
    print("Model saved to coolsense_model.pt")

    return model, X_mean, X_std


def load_model(path='coolsense_model.pt'):
    """Load trained model for inference."""
    checkpoint = torch.load(path, weights_only=False)
    model = CoolSenseOptimizer()
    model.load_state_dict(checkpoint['model_state'])
    model.eval()
    return model, checkpoint['X_mean'], checkpoint['X_std']


def neural_network_optimize(state, model, X_mean, X_std):
    """
    Given current system state from the digital twin, return recommended
    control settings and human-readable reasoning.
    """
    n = state['nodes']

    # Build 16-feature input vector (same order as training)
    input_vec = np.array([
        n['node1']['ph'], n['node1']['ec'], n['node1']['turbidity'],
        n['node2']['ph'], n['node2']['ec'], n['node2']['turbidity'],
        n['node3']['ph'], n['node3']['ec'], n['node3']['turbidity'],
        n['node4']['ph'], n['node4']['ec'], n['node4']['turbidity'],
        state['coc'],
```

```
    state['corrosion']['corrosion_severity_index'],
    state['corrosion']['cu_ppb'],
    state['biocide']['total_chlorine_mgL'],
], dtype=np.float32)

# Normalize using training statistics
input_norm = (input_vec - X_mean) / (X_std + 1e-8)
input_tensor = torch.FloatTensor(input_norm).unsqueeze(0)

# Get recommendation
with torch.no_grad():
    output = model(input_tensor)[0]

recommendations = {
    'recommended_blowdown_rate_m3_day': round(output[0].item(), 1),
    'recommended_biocide_dose_mgL': round(output[1].item(), 2),
    'recommended_ph_setpoint': round(output[2].item(), 2),
    'recommended_coc_target': round(output[3].item(), 1),
    'reasoning': [],
}

# --- Generate human-readable reasoning ---
cu = state['corrosion']['cu_ppb']
if cu > 50:
    new_cu = 10 ** (-0.5 * recommendations['recommended_ph_setpoint'] + 6.0)
    recommendations['reasoning'].append(
        f"Copper estimated at {cu:.0f} ppb (permit limit: 18 ppb). "
        f"Raising pH to {recommendations['recommended_ph_setpoint']:.1f} "
        f"reduces copper solubility to ~{new_cu:.0f} ppb."
    )
elif cu > 18:
    recommendations['reasoning'].append(
        f"Copper at {cu:.0f} ppb exceeds 18 ppb permit limit. "
        f"pH adjustment and blowdown increase recommended."
    )

biocide = state['biocide']
if biocide['effective_hocl_mgL'] < 0.2:
    recommendations['reasoning'].append(
        f"Effective biocide (HOCl) at {biocide['effective_hocl_mgL']:.2f} mg/L "
        f"is below 0.2 mg/L minimum for microbial control. "
        f"Increasing dose to {recommendations['recommended_biocide_dose_mgL']:.1f} mg/L."
    )
elif biocide['total_chlorine_mgL'] > 2.0:
    recommendations['reasoning'].append(
        f"Biocide residual at {biocide['total_chlorine_mgL']:.1f} mg/L — "
        f"high discharge risk. Reducing dose to "
```

```
        f"{recommendations['recommended_biocide_dose_mgL']:.1f} mg/L."
    )

coc = state['coc']
if coc > 6:
    recommendations['reasoning'].append(
        f"Cycles of concentration at {coc:.1f} — contaminants highly "
        f"concentrated. Recommending CoC target of "
        f"{recommendations['recommended_coc_target']:.1f} with blowdown at "
        f"{recommendations['recommended_blowdown_rate_m3_day']:.0f} m³/day."
    )

corrosion = state['corrosion']
if corrosion['corrosion_flag']:
    recommendations['reasoning'].append(
        f"Turbidity spike between basin and heat exchanger "
        f"(severity: {corrosion['corrosion_severity_index']:.2f}). "
        f"Active corrosion detected in piping. Physical inspection recommended."
    )

if not recommendations['reasoning']:
    recommendations['reasoning'].append(
        "System operating within normal parameters. Minor optimizations applied."
    )

return recommendations
```

5.4 Why This Works

- **Training takes ~2 minutes** on any laptop. 5000 scenarios × small grid search × small network.
- **Inference takes ~1 ms.** Real-time, no lag on the dashboard.
- **No real-world data needed.** The digital twin generates all training data synthetically. This is a massive advantage — you never have to touch a real cooling tower.
- **Outputs are physically interpretable.** Each output maps to a real control knob (blowdown rate, biocide dose, pH, CoC), not an abstract number.
- **The reasoning is generated from the physics, not the NN.** The NN just picks the numbers; the reasoning strings come from the digital twin's sub-models. This means the reasoning is always physically grounded, not a hallucination.

---

6. THE MAIN LOOP — TYING IT ALL TOGETHER

```python
```

```python
# main.py — CoolSense main loop

import time
import json
import math
from simulator import CoolingTowerSimulator
from neural_network import load_model, neural_network_optimize

# --- Configuration ---

MAKEUP_CHEMISTRY = {
    'calcium_mgL': 80,
    'magnesium_mgL': 30,
    'silica_mgL': 15,
    'chloride_mgL': 50,
    'sulfate_mgL': 100,
}

PERMIT_LIMITS = {
    'cu_ppb': 18,
    'zn_ppb': 120,
    'total_chlorine_mgL': 0.1,
    'ph_min': 6.5,
    'ph_max': 9.0,
    'turbidity_ntu': 50,
}

BLOWDOWN_FLOW_M3_DAY = 50
POLL_INTERVAL = 2  # seconds


def run_digital_twin(nodes, makeup_chemistry, permit_limits, blowdown_flow,
                     biocide_dose=2.0, time_since_dose=2.0, temperature=35.0):
    """
    Run all 4 digital twin sub-models on the current sensor data.
    Returns complete system state.
    """
    from neural_network import compute_pollution_score  # reuse the physics

    # Sub-model 1: Concentration
    coc = nodes['node2']['ec'] / nodes['node1']['ec'] if nodes['node1']['ec'] > 0 else 1.0
    basin_chemistry = {k: v * coc for k, v in makeup_chemistry.items()}

    # Sub-model 2: Corrosion
    ph = nodes['node3']['ph']
    cu_ppb = 10 ** (-0.5 * ph + 6.0) * (2 ** ((temperature - 25) / 20))
```

```
cu_ppb = max(5, min(cu_ppb, 1000))
zn_ppb = 10 ** (-0.4 * ph + 5.5) * (2 ** ((temperature - 25) / 20))
zn_ppb = max(10, min(zn_ppb, 500))

turb_2 = nodes['node2']['turbidity']
turb_3 = nodes['node3']['turbidity']
corrosion_severity = (turb_3 - turb_2) / turb_2 if turb_2 > 0 else 0

if corrosion_severity > 0.5:
    source = "SEVERE corrosion in heat exchanger / piping"
elif corrosion_severity > 0.2:
    source = "Moderate corrosion in heat exchanger segment"
elif corrosion_severity > 0.05:
    source = "Minor corrosion — normal range"
else:
    source = "No significant corrosion"

corrosion = {
    'cu_ppb': round(cu_ppb, 1),
    'zn_ppb': round(zn_ppb, 1),
    'corrosion_severity_index': round(max(0, corrosion_severity), 3),
    'corrosion_flag': corrosion_severity > 0.2,
    'source_diagnosis': source,
}

# Sub-model 3: Biocide
k = 0.2 * (1.05 ** (temperature - 25)) * (10 ** (0.1 * (nodes['node2']['ph'] - 7)))
total_cl2 = biocide_dose * math.exp(-k * time_since_dose)
hocl_frac = 1 / (1 + 10 ** (nodes['node2']['ph'] - 7.54))

biocide = {
    'total_chlorine_mgL': round(total_cl2, 3),
    'effective_hocl_mgL': round(total_cl2 * hocl_frac, 3),
    'hocl_fraction_pct': round(hocl_frac * 100, 1),
}

# Sub-model 4: Blowdown compliance
exceedances = []
discharge = {
    'cu_ppb': cu_ppb, 'zn_ppb': zn_ppb,
    'total_chlorine_mgL': total_cl2,
    'ph': nodes['node4']['ph'],
    'turbidity_ntu': nodes['node4']['turbidity'],
}
checks = [
    ('Copper', 'cu_ppb', cu_ppb, permit_limits.get('cu_ppb', 9999)),
    ('Zinc', 'zn_ppb', zn_ppb, permit_limits.get('zn_ppb', 9999)),
```

```
        ('Chlorine', 'total_chlorine_mgL', total_cl2, permit_limits.get('total_chlorine_mgL', 9999)),
        ('Turbidity', 'turbidity_ntu', nodes['node4']['turbidity'], permit_limits.get('turbidity_ntu', 9999)),
    ]
    for name, key, val, limit in checks:
        if val > limit:
            exceedances.append({
                'parameter': name, 'value': round(val, 2),
                'limit': limit,
                'severity_pct': round((val / limit - 1) * 100, 1),
            })

    return {
        'nodes': nodes,
        'coc': round(coc, 2),
        'basin_chemistry': basin_chemistry,
        'corrosion': corrosion,
        'biocide': biocide,
        'discharge': discharge,
        'exceedances': exceedances,
        'compliant': len(exceedances) == 0,
        'timestamp': time.time(),
    }


def main():
    # Initialize simulator
    sim = CoolingTowerSimulator()

    # Load trained neural network
    model, X_mean, X_std = load_model('coolsense_model.pt')

    # History for dashboard charts
    history = []

    print("=" * 60)
    print("  CoolSense — Cooling Tower Pollution Minimization System")
    print("=" * 60)
    print()

    tick = 0
    while True:
        # 1. GET SENSOR DATA (from simulator — swap for real sensors later)
        nodes = sim.read_all_nodes()
        sensor_data = {
            'node1': nodes['node1'],
            'node2': nodes['node2'],
            'node3': nodes['node3'],
```

```
    'node4': nodes['node4'],
}

# 2. RUN DIGITAL TWIN
state = run_digital_twin(
    sensor_data, MAKEUP_CHEMISTRY, PERMIT_LIMITS,
    BLOWDOWN_FLOW_M3_DAY,
    biocide_dose=sim.biocide_dose,
    time_since_dose=sim.time_since_biocide,
)

# 3. RUN NEURAL NETWORK
recommendations = neural_network_optimize(state, model, X_mean, X_std)

# 4. APPLY RECOMMENDATIONS TO SIMULATOR (closes the loop)
sim.apply_recommendations(recommendations)

# 5. STORE HISTORY
history.append({
    'timestamp': state['timestamp'],
    'coc': state['coc'],
    'cu_ppb': state['corrosion']['cu_ppb'],
    'zn_ppb': state['corrosion']['zn_ppb'],
    'chlorine_mgL': state['biocide']['total_chlorine_mgL'],
    'turbidity_node4': sensor_data['node4']['turbidity'],
    'ph_node4': sensor_data['node4']['ph'],
    'compliant': state['compliant'],
    'recommendations': recommendations,
})
# Keep last 500 data points (~16 minutes at 2s interval)
history = history[-500:]

# 6. PRINT STATUS (replace with dashboard push)
if tick % 5 == 0:  # print every 10 seconds
    status = "✅ COMPLIANT" if state['compliant'] else "🔴 EXCEEDANCE"
    print(f"\n[{time.strftime('%H:%M:%S')}] {status}")
    print(f"  CoC: {state['coc']:.1f} | "
          f"Cu: {state['corrosion']['cu_ppb']:.0f} ppb | "
          f"Zn: {state['corrosion']['zn_ppb']:.0f} ppb | "
          f"Cl₂: {state['biocide']['total_chlorine_mgL']:.2f} mg/L")
    if state['exceedances']:
        for exc in state['exceedances']:
            print(f"  ⚠ {exc['parameter']}: {exc['value']} "
                  f"(limit: {exc['limit']}, +{exc['severity_pct']}%)")
    print(f"  NN recommends: blowdown={recommendations['recommended_blowdown_rate_m3_day']}m³/d "
          f"| biocide={recommendations['recommended_biocide_dose_mgL']}mg/L "
          f"| pH={recommendations['recommended_ph_setpoint']} "
```

```
            f"| CoC={recommendations['recommended_coc_target']}")
        for reason in recommendations['reasoning']:
            print(f"  → {reason}")

    tick += 1
    time.sleep(POLL_INTERVAL)


if __name__ == '__main__':
    main()
```

---

7. THE DASHBOARD — WHAT IT SHOWS

7.1 Layout (Single Screen, 4 Panels)

```
┌──────────────────────────────────────────────────────────────────────────┐
│ COOLSENSE — Cooling Tower Pollution Minimization System                  │
├──────────────────────────────────┬───────────────────────────────────────┤
│                                  │                                       │
│ PANEL 1: LIVE SENSOR DATA        │ PANEL 2: DIGITAL TWIN                 │
│                                  │                                       │
│ 4 nodes, each showing            │ Visual diagram of cooling loop        │
│ pH / EC / Turbidity              │ with each segment color-coded:        │
│                                  │ Green = clean                         │
│ Color-coded:                     │ Yellow = elevated                     │
│ Green / Yellow / Red             │ Red = exceeding threshold             │
│                                  │                                       │
│ Shows DELTAS between nodes:      │ Shows INFERRED values:                │
│ "Corrosion adding 17 NTU         │ Estimated Cu: XX ppb                  │
│  turbidity in HX segment"        │ Estimated Zn: XX ppb                  │
│                                  │ Biocide residual: X.X mg/L            │
│                                  │ CoC: X.X                              │
│                                  │ Compliance: YES/NO                    │
│                                  │                                       │
├──────────────────────────────────┼───────────────────────────────────────┤
│                                  │                                       │
│ PANEL 3: TREND CHARTS            │ PANEL 4: AI RECOMMENDATIONS           │
│                                  │                                       │
│ Time-series (last 10 min):       │ Current recommendations:              │
│ - Estimated Cu (ppb)             │                                       │
│ - Biocide residual (mg/L)        │ ► Blowdown: 65 m³/day                 │
│ - CoC                            │   (currently 50 — INCREASE)           │
│ - Turbidity at blowdown          │ ► Biocide: 1.8 mg/L                   │
│                                  │   (currently 2.5 — DECREASE)          │
```

```
│ Red dashed lines = permit    │ ► pH setpoint: 8.4          │
│ limits                       │   (currently 8.0 — RAISE)   │
│                              │ ► CoC target: 4.5           │
│ Prediction line (dotted)     │   (currently 5.2 — LOWER)   │
│ extending into the future    │                             │
│                              │ REASONING:                  │
│                              │ "Cu at 72 ppb exceeds permit.  │
│                              │  Raising pH to 8.4 reduces Cu  │
│                              │  solubility to ~22 ppb.        │
│                              │  Combined with blowdown        │
│                              │  increase, predicted discharge │
│                              │  Cu drops to 15 ppb."          │
│                              │                             │
└──────────────────────────────┴─────────────────────────────┘
```

7.2 Recommended Technology

Streamlit — fastest to build, looks good enough for a demo, pure Python (no frontend code).

Key components:

- `st.metric()` for live readings with delta arrows
- `st.line_chart()` or Plotly for time-series
- `st.columns()` for the 4-panel layout
- `st.error()` / `st.warning()` / `st.success()` for compliance status
- Auto-refresh every 2 seconds using `streamlit-autorefresh` package

7.3 Demo Event Controls

Add a sidebar with buttons to trigger simulated events:

```python
```

```
# In the Streamlit dashboard:
st.sidebar.header("Demo Event Controls")

if st.sidebar.button("🔧 Corrosion Spike"):
    sim.inject_event('corrosion_spike', magnitude=1.0)

if st.sidebar.button("🦠 Biocide Overdose"):
    sim.inject_event('biocide_overdose', magnitude=1.0)

if st.sidebar.button("🦠 Biocide Underdose"):
    sim.inject_event('biocide_underdose', magnitude=1.0)

if st.sidebar.button("🏜️ Drought (Makeup Quality Drop)"):
    sim.inject_event('makeup_quality_drop', magnitude=1.0)

if st.sidebar.button("📈 Concentration Buildup"):
    sim.inject_event('concentration_buildup', magnitude=1.0)

if st.sidebar.button("⬇️ pH Drop"):
    sim.inject_event('ph_drop', magnitude=1.0)

if st.sidebar.button("🌊 Turbidity Event"):
    sim.inject_event('turbidity_event', magnitude=1.0)
```

Demo flow:

1. System starts in steady state. Dashboard is green. NN recommends minor optimizations.
2. Click "Corrosion Spike" — turbidity surges at Node 3, Cu estimate jumps, dashboard goes red, NN recommends pH raise + blowdown increase, reasoning explains why.
3. System responds to NN recommendations — watch Cu come back down, dashboard returns to green.
4. Click "Biocide Overdose" — chlorine spikes, NN recommends dose reduction.
5. Click "Drought" — makeup quality degrades, CoC effectively increases, NN adjusts all four controls simultaneously.

Each event is a self-contained story: problem → diagnosis → recommendation → resolution.

---

8. FILE STRUCTURE

```
coolsense/
├── simulator.py          # Cooling tower sensor simulator
├── neural_network.py     # NN definition, training, inference
├── main.py               # Main loop: simulator → twin → NN → output
```

```
├── train.py            # Script to train the NN (run once)
├── dashboard.py        # Streamlit dashboard
├── coolsense_model.pt  # Trained model file (generated by train.py)
└── README.md
```

8.1 How to Run

```bash
# Step 1: Install dependencies
pip install torch numpy streamlit streamlit-autorefresh plotly

# Step 2: Train the neural network (run once, takes ~2 minutes)
python train.py

# Step 3: Run the system (terminal output mode)
python main.py

# Step 4: Run the dashboard (visual mode)
streamlit run dashboard.py
```

train.py:

```python
from neural_network import train_model
train_model(n_scenarios=5000, n_epochs=200)
```

---

9. WHAT TO BUILD AND IN WHAT ORDER

Phase 1: Get the simulator running

1. Implement `simulator.py`
2. Test: call `read_all_nodes()` in a loop and print the values
3. Test: call `inject_event()` and verify values change realistically
4. Time estimate: a few hours

Phase 2: Get the digital twin running

1. Implement the 4 sub-models in `main.py` (or as separate functions)
2. Test: feed simulator data through the twin, verify inferred values make sense
3. Verify: corrosion severity spikes when you inject a corrosion event

4. Verify: Cu estimate rises when pH drops
5. Time estimate: a few hours

Phase 3: Train the neural network

1. Implement `neural_network.py`
2. Run `train.py` — should complete in ~2 minutes
3. Test: feed a known state through the NN and verify the recommendation is physically sensible
4. Time estimate: a few hours

Phase 4: Close the loop

1. Wire up `main.py`: simulator → twin → NN → `apply_recommendations()` → simulator
2. Run and verify the system stabilizes (NN recommendations should drive the system toward compliance)
3. Inject events and watch the NN respond
4. Time estimate: a couple of hours

Phase 5: Build the dashboard

1. Implement `dashboard.py` in Streamlit
2. Add the 4 panels
3. Add event control buttons in the sidebar
4. Polish colors, labels, layout
5. Time estimate: a few hours to a day depending on polish

Phase 6: Rehearse the demo

1. Plan which events to trigger and in what order
2. Practice narrating the three-act structure
3. Record a backup video